American University of Armenia

College of Science and Engineering

Artificial Intelligence Project: Gomoku

Aleks Mantashyan, Hayk Grigoryan, Hayk Shahsuvaryan, Rita Chamiyan

Fall, 2025

Abstract 4

Chapter 1. Introduction 5

1.1 What Gomoku Is 5

1.2 Goal of This Project 5

Chapter 2. Search Complexity & Constraints 7

2.1 Branching Factor in Early vs. Mid Game 7

2.2 Possible States 7

2.3 Exponential Growth of Game Trees 7

2.4 Move Filtering, Practical Time and Depth Limits 8

Chapter 3. Literature Review 9

3.1 Minimax 9

3.2 Alpha–Beta Pruning 9

3.3 Pattern-Based Evaluation 9

3.4 Threat-Based Search 9

3.5 Monte Carlo Tree Search (MCTS) 10

3.6 Neural Network Methods (AlphaZero Style) 10

Chapter 4: Methodology 11

4.1 Random Agent 11

4.2 Heuristic Minimax Agent 11

4.3 Alpha-Beta Agent 11

Chapter 5: Search Strategy Constraints and Design Decisions 12

5.1 Heuristic design 12

5.2 Candidate move strategy 15

5.3 System Architecture 15

Chapter 6. Evaluation Plan 16

6.1 Metrics 16

1. Win Rate Metric 16

2. Average Move Time Metric 16

3. Number of Moves Metric 16

6.2 Number of Games Played 17

Chapter 7: Results 18

Conclusion 23

References..... **24**

Abstract

Many decades ago, professional Gomoku players believed that whoever started first would always win if both players found optimal moves over the whole game. So far, there is no evidence to confirm that. Finding an optimal move is hard due to the huge state space of the problem. In this project, we want to compare three types of agents — Random, Heuristic and Depth Limited Minimax, to see how different strategies influence the effectiveness and quality of the game.

Chapter 1. Introduction

1.1 What Gomoku Is

Gomoku is a traditional strategic board game played on a 15×15 grid. Two players, Black and White, take turns placing stones. Black always moves first under the standard rules. Although there exist several starting rule variations, this project considers only the classical starting method.

The goal of the game is to be the first player to form an unbroken line of five stones horizontally, vertically, or diagonally. The other way to end the game is to fill the full board , which will be considered as a draw. Gomoku demands strong tactical planning and long-term strategic thinking. The game's large board size and high number of moves available at each turn make the game rather challenging.

1.2 Goal of This Project

The goal of this project is to implement and experimentally compare several Gomoku-playing agents. We focus on three types of agents: a Heuristic Minimax agent, an Alpha-Beta–enhanced Minimax agent, and the simplest one - Random agent.
Besides simple win-rate evaluations, our analysis also examines computational costs, involving the number of nodes expanded and pruning cutoffs, as well as tracking average time taken per move at different stages of the game.

More specifically, this project aims to:

1. Measure playing performance by comparing win rates between agents with different search depths and algorithms.
2. Evaluate computational cost by collecting statistics such as nodes expanded, number of cutoffs, and average time per move in different stages of the game.
3. Study the effect of search depth and determine whether deeper search always leads to better moves.
4. Examine the impact of move filtering and heuristics on reducing the branching factor and improving decision-making.

To improve search performance, we incorporate move filtering and use a pattern-based heuristic evaluation function that restricts the branching factor, making search much more efficient. Each agent's performance is directly affected by different depths, heuristics, and pruning strategies, so by systematically analyzing

those, we aim to understand how algorithmic modifications result in differences in playing strength and computational efficiency.

Overall, these objectives help us understand the strengths and weaknesses of classical search techniques when applied to a complex board game like Gomoku.

Chapter 2. Search Complexity & Constraints

2.1 Branching Factor in Early vs. Mid Game

Gomoku's board is completely empty in the opening stage of the game, so each player has 225 possible moves. The second player still has 224 moves and even after several moves, there are often more than 200 legal moves left.

2.2 Possible States

The board has 225 intersections, and each spot can be in one of three states: empty, black, or white. A rough upper bound on all possible arrangements is:

$3^{225} \approx 10^{107}$

Even if only 1% of these board states are reachable through legal play, it still leaves a gigantic space.

2.3 Exponential Growth of Game Trees

Search trees grow exponentially, not linearly. One extra move of depth multiplies the total nodes by the branching factor.

To see how extreme this gets, imagine the AI wants to look 4 moves ahead (which is very shallow in board-game AI terms).
With 50 possible moves per turn:

$50^4 \approx 6,250,000$ board evaluations

Even if the program evaluates 100,000 positions per second (which is very fast for Python), this still takes 62.5 seconds — meaning one move takes more than a minute.
For a game that requires 30–60 moves, this becomes completely unplayable.This is why we cannot simply "search deeper" the way we do in small games.

2.4 Move Filtering and Depth Limits

Because of all these constraints, our agent cannot rely on full search. Instead, it must use smarter and lighter strategies.In our project, we used depth limits that can be changed by the user and move filtering.

The aim of depths is to decide how many moves ahead do we want our agent to look at. We mainly used one, two and three depths as beyond that process takes unreasonable time to be computed.

Another improvement is to restrict the number of legal moves by allowing moves that are within 2 cells of existing stones or part of important patterns. This reduces the branching factor up to ten times and makes search possible. The complexity becomes:

- Depth 1: ~20 moves
- Depth 2: ~400 moves
- Depth 3: ~8,000 moves

Move filtering is both safe and necessary. It is safe because strong Gomoku gameplay rarely requires placing stones far away from existing patterns. Important threats and structures form near the current group of stones, so filtering out distant moves rarely removes good options.It is necessary because without filtering, even depth 2 or 3 would take too long to compute. Filtering enables the agent to search deeper and focus on meaningful positions rather than wasting time on irrelevant parts of the board.

Together, depth limits and move filtering transform an otherwise intractable search problem into something manageable while still preserving strategic quality.

Chapter 3. Literature Review

AI for board games has been studied for many years, and Gomoku benefits from many of the techniques originally developed for other games trying to deal with the main challenge: the search space, time limit.

3.1 Move filtering

Move filtering is a common approach in games with huge state space, Gomoku isn't an exception. In this game filtering of legal moves is based on already placed stones. The legal moves are restricted to be placed a maximum of 2 square distance from stones that are already on the board. (Petratti, 2023)

3.2 Minimax

Minimax is the basic algorithm used in many games. It only considers the next move, while the improved version - Heuristic Minimax - looks ahead a few moves assuming both players play perfectly and picks the move that leads to the best final outcome for the player.
Although minimax is simple to understand, it becomes slow very quickly in Gomoku because there are too many possible moves. Still, it is the foundation of many other techniques.

3.3 Alpha–Beta Pruning

Alpha–beta pruning improves minimax by skipping moves that cannot change the final decision. This makes the search much faster and allows the AI to look deeper into the future.

3.4 Pattern-Based Evaluation

Since Gomoku has too many possible moves, AI agents must evaluate board positions using specific patterns. It is the main heuristic (with some variations) used for solving this game. It can be used for depth-limited search.
Common patterns include: open three (a threat in two directions), open four (must-block threat), double threats, closed fours and other near-winning shapes. Recent programs work to learn patterns automatically. Pattern evaluation is a key part of almost every strong Gomoku AI.

3.5 Threat-Based Search

Competitive Gomoku programs use threat-based search, which focuses only on moves that create immediate threats, instead of checking the whole board. This method can find long forced winning sequences that normal minimax would miss.

3.6 Monte Carlo Tree Search (MCTS)

Monte Carlo Tree Search does not evaluate every move according to the rules of the given game, rather, it plays many simulations of the game and learns which moves tend to have better outcomes. MCTS works well in large games, but it is used more for stochastic games. MCTC relies on random playouts (Russell & Norvig, 2021, p. 207), however, Gomoku is a precise game and needs particular sequences for success. Winning may be one move away but MCTC with its random choices rarely will find that move. In addition to that, traditional MCTC does not use heuristics, however, without pattern recognition it will not be possible to solve this game.

3.7 Neural Network Methods (AlphaZero Style)

More recent AI systems, such as AlphaZero use the method of mixing search with deep learning. Those new systems play millions of games against themselves and use this to learn important patterns. In famous games like Go, Sogi and Chess, AlphaZero achieved very impressive results, winning over strong engines like Stockfish and Elmo.

Chapter 4: Methodology

4.1 Random Agent

A random agent is the simplest type of agent that chooses a move without considering any information, it simply selects a legal move uniformly at random with no evaluation. Such an agent is very fast and will always reach a terminal state. However, it is highly ineffective as a simple reflex agent and can be easily defeated, because its moves are not based on any strategic and meaningful analysis of a particular game state.

4.2 Heuristic Minimax Agent

The Heuristic Minimax Agent with depth limit uses a minimax algorithm; however, it looks at some k steps ahead before choosing the next move. Let's say if depth is 3, it looks at its next best move, then predicts the opponent's best response and also takes into consideration its best reply to the opponent's response. When the depth is reached at leaf nodes, the position is evaluated based on the heuristic function.

By looking k depths ahead, the agent gains the advantage of finding traps and blocking possible dangers. It assumes the opponent plays optimally and makes a move respectively. If the opponent doesn't, it plays as optimally as before, if not more.

Overall, the idea is to find a move with the best possible heuristic evaluation value after k simulated interactions.

4.3 Alpha-Beta Agent

Alpha-beta is optimisation for the minimax algorithm, which will be applied in the Heuristic Minimax agent to prune away branches that cannot affect the final decision. Alpha-Beta agent is giving the same effectiveness as heuristic Minimax agent (with the same depths), however, it does computations faster. With depth 1, alpha-beta agent does not prune anything, hence it gives the same number of explored nodes as Heuristic minimax agent with depth 1.

Chapter 5: Search Strategy Constraints and Design Decisions

Gomoku has a huge state space; hence, taking into consideration all possible moves is not realistic. Therefore, we need to make some decisions that will make implementation practically manageable.

One of the most popular approaches that we used is search with fixed depth k. Evaluating the whole tree at once is impossible; however, comparing values of 2 to 3 depth layers makes that task achievable.

For further improvements, we evaluate the heuristics of potential moves and arrange them by how promising they are. Those paths are explored first. This gives space for alpha-beta pruning to be much more effective, since weaker branches are pruned earlier.

Some advanced techniques like Threat-Space Search, Monte Carlo Tree Search, neural network evaluation, and perfect-play databases are excluded due to their high complexity and computational cost.

5.1 Heuristic design

The strength of our Gomoku agent depends largely on the design of its heuristic evaluation function, which estimates how favourable a position we have during that phase of the game. As the gomokus board has a large branching factor, the minimax algorithm can not explore all the continuations of the game; that’s why we need to have an evaluation function that will tell us which moves can be dangerous and which are winning positions.

Our heuristic is constructed around identifying key positions in every Gomoku game. They are five in a row: open fours, closed fours, open threes, closed threes, and open twos. Five in a row represents an immediate win for the agent; that’s why it will have the highest possible score.

Then comes open four, which is an extremely dangerous layout, and even if one side is defended, the other side has a winning move. That's why it will receive the highest weight after 5 in a row.

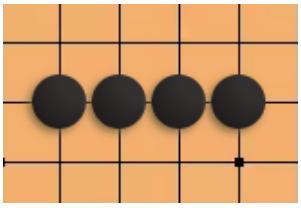

Figure 5.1: Open four

The next configuration is the second best one. It is an immediate win in agents' move but can be blocked if it is the opponent's move. However, in case it is blocked, it has potential to be expanded from left or right sides. That is the reason why is it better than closed 4.

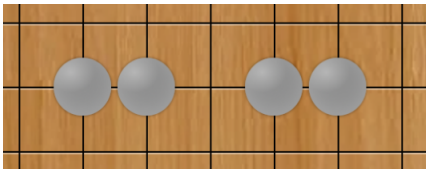

Figure 5.2: Broken 4

Closed 4 is not very dangerous, as it can be blocked by one turn. But it is a guaranteed win if the agent moves.

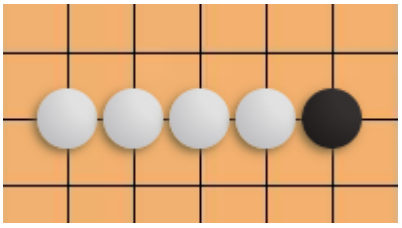

Figure 5.3: Closed 4

Next one is broken 3, it is the best configuration with 3 stones. It is a guaranteed win in agents' move as it creates open 4, in case it is blocked, it has the possibility to be expanded from either side.

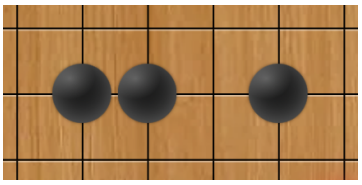

Figure 5.4: Broken 3

Open 3 has the potential to grow to become closed 4, broken 4 or open 4, receiving moderate weight, while closed threes and twos receive smaller values representing the early game stage and no threat.

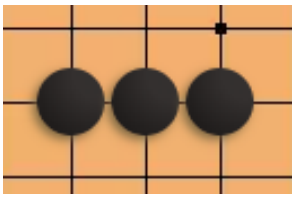

Figure 5.5: Open 3

Closed 3 is the least valued configuration in our evaluation table. It has almost no threat when playing against any agent with minimal defence but it’s better than nothing

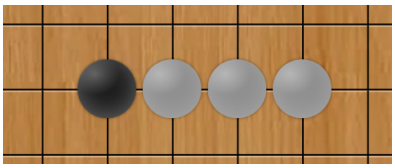

Figure 5.6: Closed 3

The evaluation value is computed in the following formula.

Eval = My_score - Opponent_score

Every time it computes an evaluation value it needs to compute the opponent's value too. This allows the agent to prioritize defence over offense or visa verse when needed.

Configuration	Value
Win	1,000,000
Open four	50,000
Broken Four	10,000
Closed Four	3,000
Broken Three	2,000
Open Three	800
Closed Three	200

5.2 Candidate move strategy

Gomoku's board contains 225 possible moves, but evaluating every empty cell will make minimax computation extremely slow; that’s why we will only consider one or two cells from existing stones. Stones placed far from the active area will not have much effect, so their exclusion will not reduce the playing strength. By limiting the accessible moves, the branching factor becomes a few times smaller, making the evaluation more efficient while maintaining strong tactical awareness.

5.3 System Architecture

System architecture includes three main components: the board, the agents, and the game manager. The board is the area of the game where the stones of each player are placed. Both agents share the same interface, which requires a single function that receives the board and returns a valid move. There are three agent types: a random agent, a Heuristic Minimax agent, and an Alpha Beta agent. Their logic is different from each other, but their interaction with the board is the same. The game manager controls the game by sending the board to the agents, applying the move by the agent, switching turns, and checking for win conditions.

Chapter 6. Evaluation Plan

Since the core purpose of our project is to compare the strengths and weaknesses of agents that use different methods, it is important to have effective and relevant evaluation methods that will allow us to both compare them and understand their effectiveness individually. One straightforward way to do this is to define some metrics and use them to evaluate each of the models, then compare the results.

6.1 Metrics

1. Win Rate Metric

One natural metric for evaluating any game is the Win Rate. When we say one player is better than the other, most of the time we mean that he/she has won more games than the other one. The same applies to agents that play Gomoku. The Win Rate metric of a particular agent shows the proportion of games it wins out of all the games it played, so using this metric for all of the agents that we have will give us a fairly accurate approximation of how good that agent is.

2. Average Move Time Metric

Another important feature of an agent is the amount of time it takes for it to decide its next move and play. Of course, no one would want to play against an agent that thinks for hours before making its next move. So it is smart to use the Average Move Time metric that brings attention to this matter. This metric is especially important because the time complexities of different models may differ significantly from each other.

3. Number of Moves Metric

It is also interesting to pay attention to the aggressiveness of the agent. Maybe one agent plays it safe by choosing moves that are more likely to end up bringing advantage to it, even if the game gets very long after that. Another one, on the other hand, chooses risky and aggressive moves that bring it closer to victory faster. To measure this, the Number of Moves metric can be used, which counts the number of moves an agent made before the game ended.

6.2 Number of Games Played

To compare and gain a deep understanding of each agent's effectiveness, each agent played all the others, so we can compare each and every one of them to the others. The agents that had relatively fast computational search played each other 30 times, half starting one and the other half the other agent. Those mutchups were Alpha–Beta Agent (Depth 2) vs Heuristic Minimax Agent (Depth 1) and Alpha–Beta Agent (Depth 2) vs Heuristic Minimax Agent (Depth 2). Random agent was tested 12 times against different agents just to evaluate its speed. FInally, the most computationally expensive one - AlphaBeta Agent(depth = 2) VS AlphaBeta Agent(depth = 3)- was limited to just 16 times because of depth 3 agents' long thinking time

Chapter 7. Results

Alpha–Beta Agent (Depth 2) / Heuristic Minimax Agent (Depth 1) vs Random Agent

Random agent is the weakest of all implemented ones. As expected it lost all games by expanding 4 to 7 nodes, and playing instantly, thinking 0.00002 to 0.000031 seconds on average.

Alpha–Beta Agent (Depth 2) vs Heuristic Minimax Agent (Depth 1)

A clear difference in effectiveness and computational time is noticed, in the games of Alpha–Beta agent with depth 2 and Heuristic Minimax agent with depth 1. The Heuristic Minimax agent makes moves faster than Alpha–Beta agent does, although Alpha-Beta prunes redundant nodes, it also looks at 1 additional depth in comparison with heuristic minimax. That is why it performs slower but finds better moves.

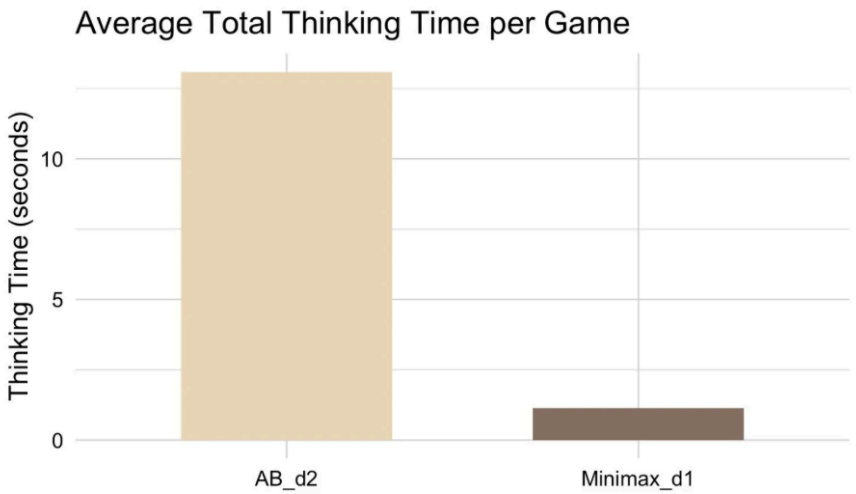

Figure 7.1

On average it took Alpha–Beta agent 13.096 seconds per game, while the Heuristic Minimax agent thought approximately 1.1 seconds. Overall, the Heuristic

Minimax agent took about 12 times less time per game than the Alpha–Beta agent.

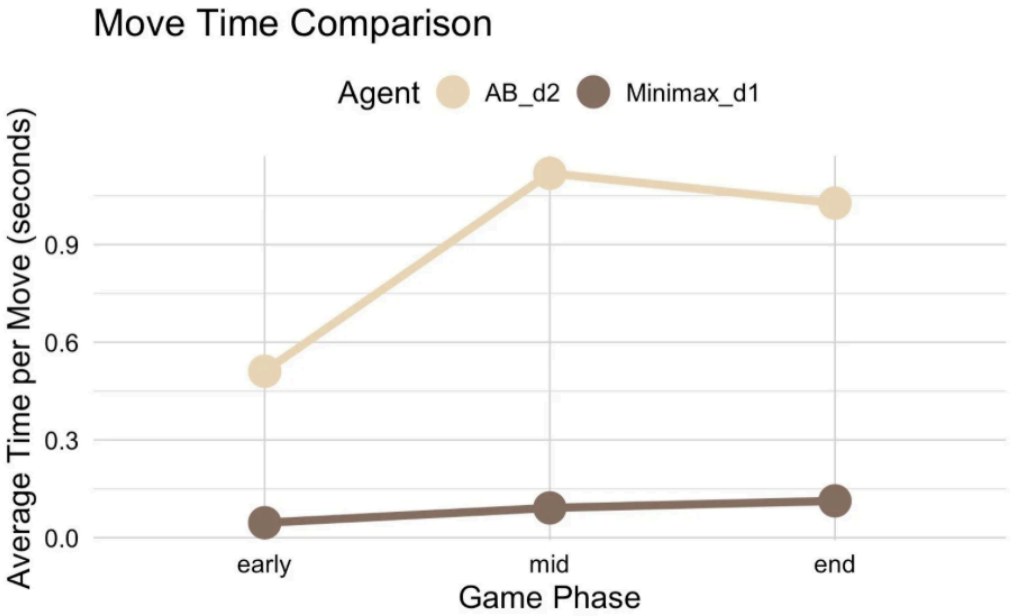

Figure 7.2

In the figure 7.2 we can see that at early stage, thinking time is faster for both agents, as less moves are available, in the midgame it spikes as the number of places stones is larger, therefore, larger is the number of filtered moves. It decreases a little at the end because less and less space is available.

AlphaBeta agent (depth = 2) VS Heuristic Minimax agent (depth = 2)

The next match is about computational performance. Booth agents work with the same search algorithm, the only difference is alpha-beta pruning optimization. On average, the AlphaBeta agent thought for 22.3 seconds of total thinking time per game, absolutely overperforming the Heuristic Minimax agent who computed around 124.8 seconds on average.

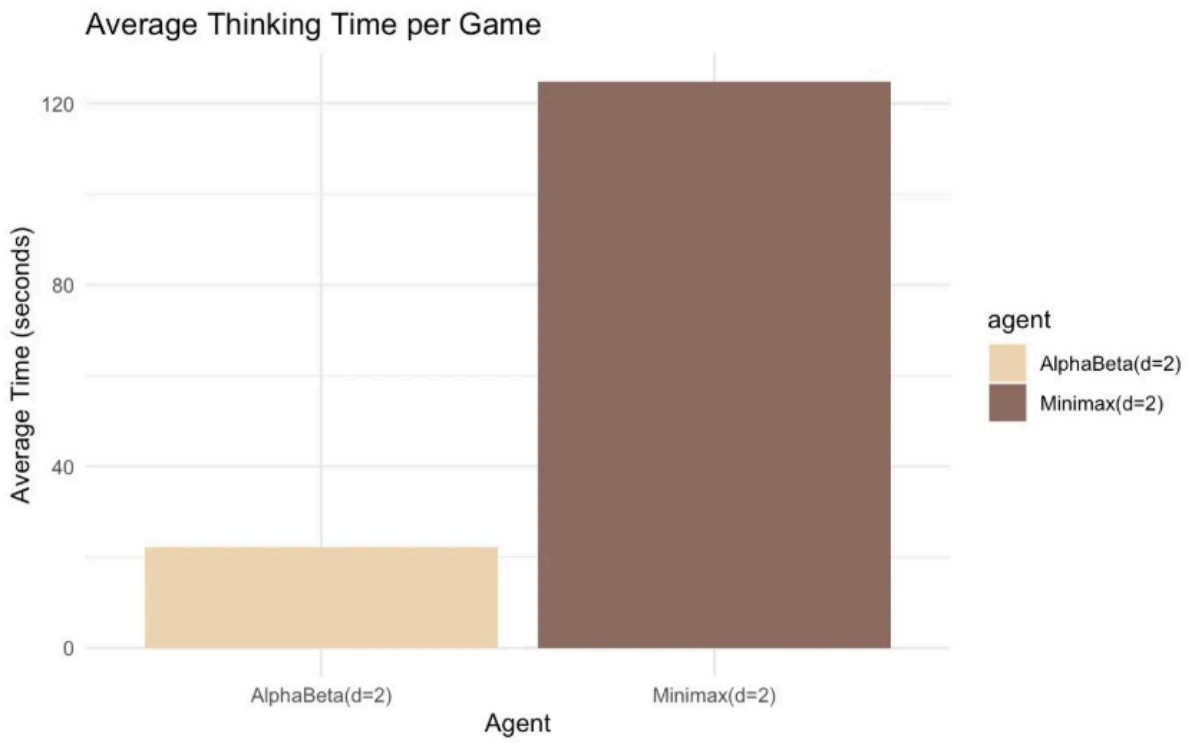

Figure 7.3

This means that Heuristic Minimax took about 5.6 times longer than AlphaBeta to complete a game. A similar pattern was observed in the number of explored nodes, AlphaBeta expanded an average of 25,349 nodes per game, which is almost 6 times less than Heuristic Minimax with its 136,696 nodes. Alpha-Beta Agent performed around 1410 cutoffs. Taking into consideration all of these, we can conclude that alpha-beta pruning is an excellent teqnique for this game.

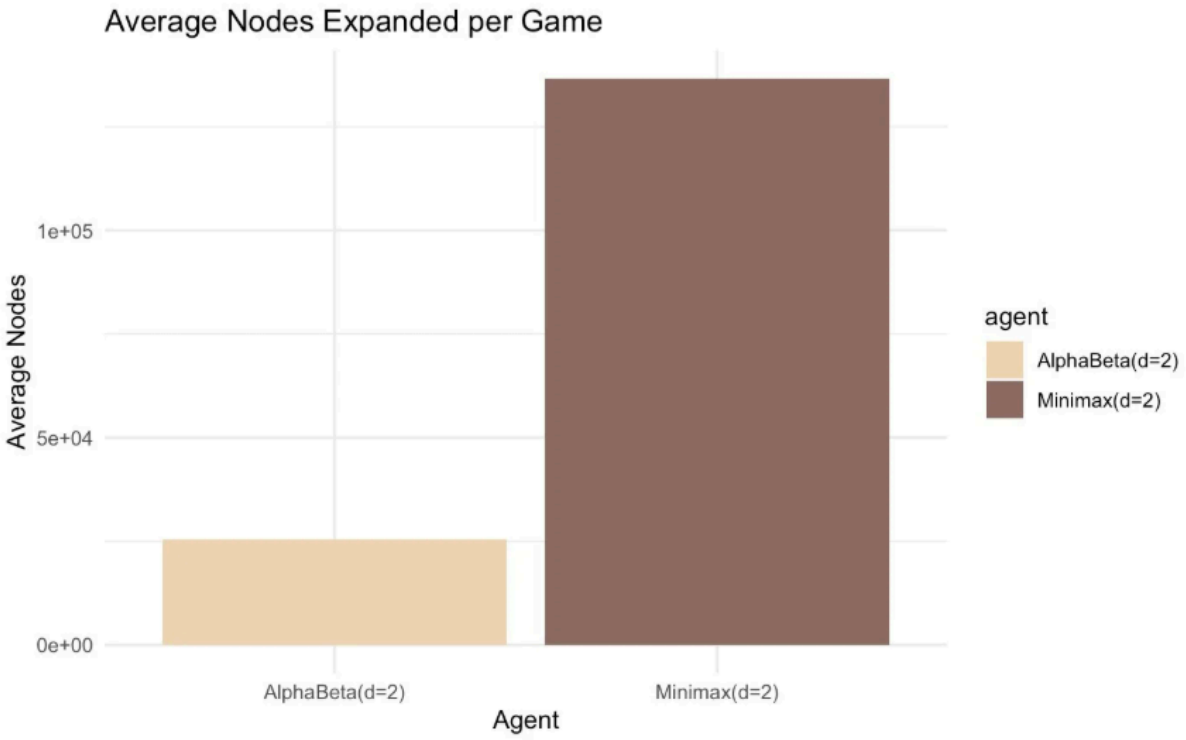

Figure 7.4

AlphaBeta won 60.0% of the games when he started, while Heuristic Minimax won 40.0%. When Heuristic Minimax played as the first player, it won 73.3% of the games, AlphaBeta won 20.0%, and 6.7% of the games ended in draws. These results show that first player advantage somehow affects the game.

AlphaBeta Agent(depth = 2) VS AlphaBeta Agent(depth = 3)

20,740 339,962 The match of depth 2 and depth 3 turned out in an unexpected way. Looking at statistics the average total thinking time per game for AlphaBeta with depth 2 was approximately 17.89 seconds, while depth 3 required about 303.93 seconds. Depth 3 has 17 times longer computational complexity per game. As expected Depth 3 expanded much more nodes than depth 2. The difference is huge, 339,962 nodes per game compared to 20,740. Obviously, the number of cutoffs increased too, approximately 10 times.

When we look at win rates, AlphaBeta with depth 2 played as the first player, it won 37.5% of the games, while depth 3 won 62.5%. However, when AlphaBeta with depth 3 played as the first player, it lost all the games and depth 2 won 100% of the games as the second player.

This was an unexpected result as depth 3 agent was expected to win as it sees 3 moves ahead, however, after considering results, conclusions were made. One of the reasons is that used heuristic is not perfect for higher depths as it limits

higher depth agents. The main cause is move filtering. Although being counter-intuitive, the agent with depth 3 might see a smaller portion of the game than depth 2. When the agent looks deeper, move filtering makes the “center” of the board shift, and because of this, it might filter off a good move.

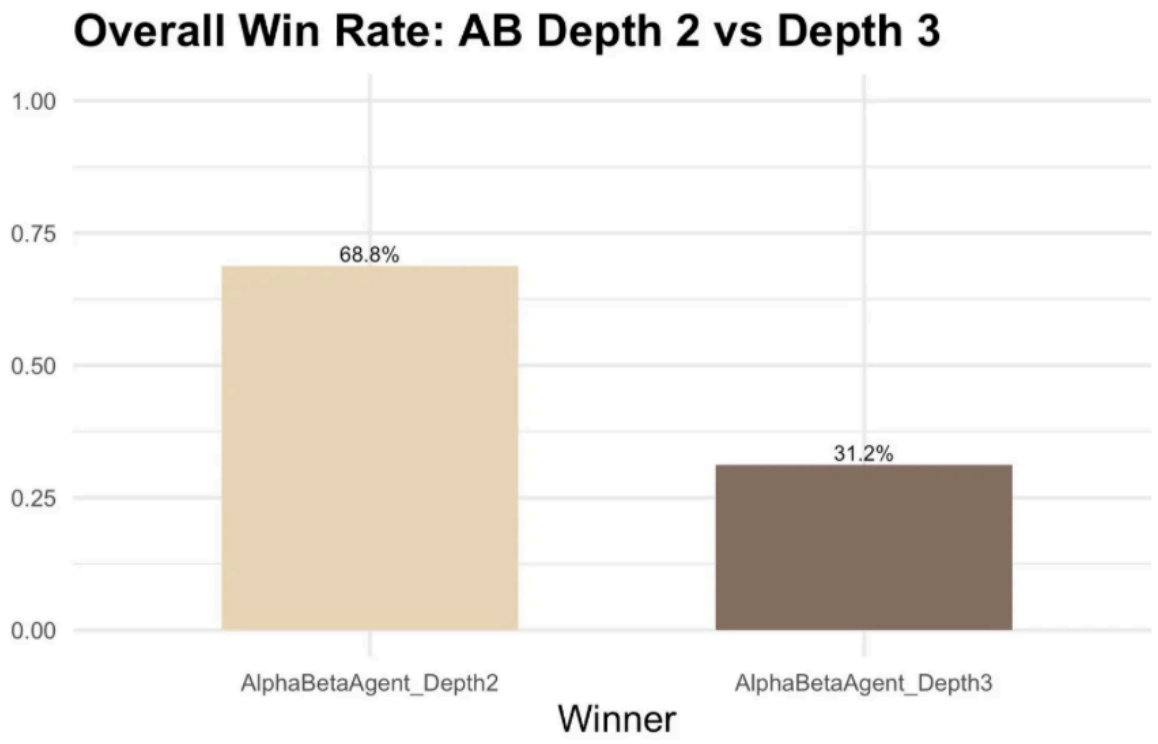

Figure 7.5

Conclusion

In conclusion, we implemented 3 types of agents, which are Random Agent, Heuristic Minimax Agent and Alpha-Beta Agent. The most optimal one for finding balance between time consumption and efficiency was Alpha-Beta Agent with depth 2. According to tests Alpha-Beta Agent (d=2) dominates Alpha-Beta Agent (d=3). The result comes from the fact that move-filtering strategy becomes too restrictive when exploring more depths(d>2). Overall, Alpha-Beta Agent (d=2) performed very well against human players too, demonstrating that precise computations can give strong results even without complex computations.

References

Petratti, V. (2023). *How we made an AI play Gomoku*. Medium. https://medium.com/@viniciuspetratti/how-we-made-an-ai-play-gomoku-5f4344d0b41

Huang, S., Hu, H., & Yang, Y. (2023). *Gomoku reinforcement learning: Benchmarking AlphaZero and MCTS with improved heuristics*. arXiv. https://arxiv.org/abs/2309.01294

Russell, S., & Norvig, P. (2021). *Artificial Intelligence: A Modern Approach, Global Edition*. Pearson Higher Ed.